Documentazione della Suite di Test — Albero di Regressione

1. Obiettivi e Ambito del Testing

Il presente documento descrive l'architettura, la matrice di tracciabilità e le schede tecniche dettagliate della suite di test automatizzata realizzata per il progetto **Albero di Regressione** (Java), in conformità al Requisito **R3** (Testing completo con JUnit e Mockito) e alle specifiche di progetto.

La suite persegue i seguenti obiettivi di qualità del software: - **Correttezza funzionale**: validazione di tutte le componenti algoritmiche (induzione dell'albero, calcolo dell'errore quadratico SSE, partizionamento discreto e continuo, predizione interattiva). - **Integrità dell'incapsulamento e polimorfismo**: verifica delle gerarchie ad oggetti (`Attribute` , `Node`) e della serializzazione binaria degli oggetti (`Serializable`). - **Resilienza e gestione delle eccezioni**: verifica di tutti i rami di errore previsti dal dominio applicativo (`TrainingDataException` , `UnknownValueException` , `DatabaseConnectionException` , `EmptySetException`). - **Integrazione reale vs. isolamento**: testing end-to-end reale su database MySQL `MapDB` (tabella `provac`) e su socket TCP (porta effimera), combinato con mock JDBC (`Mockito`) per simulare anomalie e schemi non validi senza richiedere permessi DDL speciali. - **Isolamento dell'input interattivo**: testing controllato dell'acquisizione dati da console (`Keyboard`) tramite iniezione di stream in memoria e riallineamento dello stato statico via reflection.

---

2. Tecnologie e Configurazione della Suite

La suite è integrata nel ciclo di vita Maven attraverso il plugin `maven-surefire-plugin` (versione 3.5.2) e fa uso dei seguenti framework: - **JUnit Jupiter 5.11.4** (`junit-jupiter` , `junit-bom`): supporto all'esecuzione dei test, asserzioni ricche, test annidati (`@Nested`), gestione del ciclo di vita (`@BeforeAll` , `@BeforeEach` , `@AfterEach` , `@AfterAll`) e directory temporanee sicure (`@TempDir`). - **Mockito 5.15.2** (`mockito-core` , `mockito-junit-jupiter`): creazione di mock per oggetti JDBC (`Connection` , `Statement` , `ResultSet` , `ResultSetMetaData`) e intercettazione dei costruttori tramite `Mockito.mockConstruction(DbAccess.class, ...)` .

---

3. Matrice di Tracciabilità Requisiti ↔ Test

ID Requisito	Modulo	Package	Classe di Test	M
REQ-DATA-01	`Server`	`data`	`AttributeTest`	
REQ-DATA-02	`Server`	`data`	`AttributeTest`	
REQ-DATA-03	`Server`	`data`	`AttributeTest`	
REQ-DATA-04	`Server`	`data`	`AttributeTest`	
REQ-DATA-05	`Server`	`data`	`AttributeTest`	
REQ-DATA-06	`Server`	`data`	`AttributeTest`	
REQ-DATA-07	`Server`	`data`	`DataTest`	
REQ-DATA-08	`Server`	`data`	`DataTest`	
REQ-DATA-09	`Server`	`data`	`DataTest`	
REQ-DATA-10	`Server`	`data`	`DataTest`	
REQ-DATA-11	`Server`	`data`	`DataTest`	
REQ-ERR-01	`Server`	`data`	`DataTest`	
REQ-ERR-02	`Server`	`data`	`DataTest`	
REQ-ERR-03	`Server`	`data`	`DataTest`	
REQ-ERR-04	`Server`	`data`	`DataTest`	
REQ-ERR-05	`Server`	`data`	`DataTest`	
REQ-ERR-06	`Server`	`data`	`DataTest`	

ID Requisito	Modulo	Package	Classe di Test	M
REQ-ERR-07	`Server`	`data`	`DataTest`	
REQ-DB-01	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-02	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-03	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-04	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-05	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-06	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-07	`Server`	`database`	`DatabasePackageTest`	
REQ-DB-08	`Server`	`database`	`DatabasePackageTest`	
REQ-TREE-01	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-02	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-03	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-04	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-05	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-06	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-07	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-08	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-TREE-09	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	

ID Requisito	Modulo	Package	Classe di Test	M
REQ-TREE-10	`Server`	`tree`	`TreeNodesAndAlgorithmTest`	
REQ-SOCK-01	`Server`	`server`	`SocketClientServerIntegrationTest`	
REQ-SOCK-02	`Server`	`server`	`SocketClientServerIntegrationTest`	
REQ-SOCK-03	`Server`	`server`	`SocketClientServerIntegrationTest`	
REQ-UTIL-01	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-02	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-03	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-04	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-05	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-06	`Client`	`utility`	`KeyboardTest`	
REQ-UTIL-07	`Client`	`utility`	`KeyboardTest`	
REQ-REST-01	`Server`	`server`	`EndToEndRestAndClientIntegrationTest`	
REQ-REST-02	`Server`	`server`	`EndToEndRestAndClientIntegrationTest`	
REQ-REST-03	`Server`	`server`	`EndToEndRestAndClientIntegrationTest`	
REQ-REST-04	`Server`	`server`	`EndToEndRestAndClientIntegrationTest`	
REQ-REST-05	`Server`	`server`	`EndToEndRestAndClientIntegrationTest`	
REQ-CLIENT-01	`Client`	`Client`	`TreeRestClientServiceTest`	

ID Requisito	Modulo	Package	Classe di Test	M
REQ-CLIENT-02	`Client`	`Client`	`TreeRestClientServiceTest`	
REQ-CLIENT-03	`Client`	`Client`	`TreeRestClientServiceTest`	
REQ-CLIENT-04	`Client`	`Client`	`TreeRestClientServiceTest`	
REQ-CLIENT-05	`Client`	`Client`	`TreeMermaidConverterTest`	
REQ-CLIENT-06	`Client`	`Client`	`TreeMermaidConverterTest`	
REQ-CLIENT-07	`Client`	`Client`	`TreeMermaidConverterTest`	
REQ-CLIENT-08	`Client`	`Client`	`TreeMermaidConverterTest`	
REQ-CLIENT-09	`Client`	`Client`	`TreeMermaidConverterTest`	
REQ-CLIENT-10	`Client`	`Client`	`BrowserLauncherTest`	
REQ-CLIENT-11	`Client`	`Client`	`BrowserLauncherTest`	
REQ-CLIENT-12	`Client`	`Client`	`BrowserLauncherTest`	
REQ-CLIENT-13	`Client`	`Client`	`BrowserLauncherTest`	

4. Schede Tecniche di Dettaglio per Classe di Test

4.1 `data.AttributeTest`

Questa classe valida la gerarchia di classi del package `data`: - **`testContinuousAttributeInitialization`**: verifica che il costruttore assegni correttamente nome e indice numerico, e che `toString()` restituisca il nome dell'attributo. - **`testContinuousAttributeSerialization`**: serializza su `ByteArrayOutputStream` un'istanza di `ContinuousAttribute` e la deserializza con `ObjectInputStream`, asserendo l'uguaglianza dei campi di stato. - **`testDiscreteAttributeDeduplicationAndSorting`**: fornisce valori ripetuti e disordinati (`{"Sunny", "Rain", "Sunny", "Overcast", "Rain"}`) e verifica che l'iteratore restituisca l'insieme deduplicato e ordinato lessicograficamente (`["Overcast", "Rain", "Sunny"]`), verificando inoltre che `getNumberOfDistinctValues()` restituisca `3`. - **`testEmptyAndSingleValueDomains`**: valida i casi limite di domini vuoti e a singolo elemento. - **`testDiscreteAttributeSerialization`**: valida il round-trip di serializzazione e deserializzazione preservando l'integrità del `TreeSet` interno e dell'iteratore `Iterable<String>`. - **`testPolymorphism`**: verifica l'upcasting a tipo astratto `Attribute` e il corretto dispatch dinamico.

4.2 `data.DataTest`

Questa classe testa l'acquisizione, la validazione dello schema e l'ordinamento Quicksort in-place dei dataset: - **`testLoadProvaCSuccessfully`**: effettua una connessione reale a MySQL `MapDB.provac`, verificando l'istanziazione di 15 tuple, la presenza di 2 attributi indipendenti (uno discreto `X` e uno continuo `Y`) e 1 attributo target continuo `C`. - **`testSortDiscreteAttributeFullRange`**: ordina l'intero dataset sull'attributo discreto `X` e verifica l'ordinamento in-place (tutte le 'A' prima delle 'B'). - **`testSortContinuousAttributeFullRange`**: ordina l'intero dataset sull'attributo continuo `Y` e verifica che ogni elemento sia minore o uguale al successivo. - **`testSortSubRange`**: ordina un sotto-intervallo arbitrario `[3, 8]` e verifica che le posizioni esterne rimangano inalterate mentre quelle interne risultino ordinate. - **`testSortBoundaryRanges`**: controlla i casi degenere `begin == end` e `begin > end` confermando la totale stabilità senza eccezioni. - **`testInvalidTableName`**: verifica il rigetto immediato di nomi tabella `null`, vuoti o costituiti da soli spazi con `TrainingDataException`. - **`testNonExistentTable`**: verifica la gestione dell'errore JDBC in presenza di tabella assente. - **`testTableWithLessThanTwoColumns`**: usa `Mockito.mockConstruction` su `DbAccess` per simulare un result set avente 1 sola colonna, verificando il messaggio d'errore richiesto. - **`testLastColumnNotNumeric`**: simula una tabella con target `VARCHAR`, accertando il sollevamento dell'eccezione sul tipo target. - **`testNullValuesInTuple`**: simula la presenza di un valore `null` all'interno di una tupla e verifica il blocco del caricamento. - **`testNonNumericValueInNumericColumn`**: simula una stringa all'interno di una colonna dichiarata numerica, verificando il sollevamento di `TrainingDataException`. - **`testEmptyTable`**: simula una tabella valida ma priva di righe (`rs.next() == false`), verificando l'eccezione dedicata.

4.3 `database.DatabasePackageTest`

Questa classe copre il sottosistema di persistenza relazionale: - **`testColumnProperties`**: controlla il costruttore package-private, il nome, la stringa formattata e il predicato `isNumber()` su tipi testuali e numerici. - **`testExampleAddGetIterator`**: valida la struttura dati `Example`, l'inserimento posizionale, la formattazione testuale e l'iteratore. - **`testExampleCompareTo`**: verifica la relazione d'ordine totale definita dal metodo `compareTo` su coppie di tuple identiche e distinte. - **`testInitAndCloseConnection`**: apre e chiude una connessione reale a MySQL, verificando che la connessione risulti aperta dopo `initConnection()` e chiusa dopo `closeConnection()`. - **`testTableSchemaReading`**: interroga i metadati JDBC di `provac` validando nomi colonne e tipi estratti. - **`testTableDataGetTransazioni`**: estrae le 15 tuple reali come lista di oggetti `Example`. - **`testGetDistinctColumnValues`**: esegue la query `DISTINCT` ordinata su `X` e `Y`, validando la corrispondenza dei valori univoci (`{"A", "B"}`). - **`testTableDataExceptions`**: verifica che query su tabelle non esistenti propaghino `SQLException`.

4.4 `tree.TreeNodesAndAlgorithmTest`

Questa classe valida la correttezza matematica e l'induzione algoritmica: - **`testExactSSEZeroVariance`**: sul sottoinsieme di 5 esempi con $X =' B'$, tutti con $C = 10.0$, calcola matematicamente e verifica $SSE = 0.0$ e valore predetto 10.0. - **`testExactSSEKnownSubsetA`**: sul sottoinsieme di 10 esempi con $X =' A'$ (cinque 1.0 e cinque 1.5), calcola $\bar{y} = 1.25$ ed $SSE = 0.625$, verificando la perfetta corrispondenza con l'implementazione in `LeafNode`. - **`testLeafNodeProperties`**: verifica che una foglia abbia 0 figli e la stringa inizi con `"LEAF : class="`. - **`testDiscreteNodePartitioning`**: crea un nodo di split discreto su `X`, verifica la creazione di 2 rami, la varianza aggregata di split e il comportamento di `testCondition` su valori noti e sconosciuti. - **`testContinuousNodePartitioning`**: crea un nodo di split continuo su `Y`, verificando la selezione del miglior punto di taglio e il routing dei valori. - **`testContinuousNodeConstantValue`**: gestisce il caso limite in cui l'intervallo contiene valori costanti, generando una singola partizione. - **`testTreeInductionAndPrints`**: induce l'intero albero di regressione su `provac` e verifica `printTree()` e `printRules()`. - **`testPredictClassSuccessful`**: simula la consultazione interattiva collegando stream binari in memoria (`ObjectOutputStream` / `ObjectInputStream`), rispondendo alla query e ottenendo il valore predetto atteso (10.0). - **`testPredictClassUnknownValueException`**: invia una risposta non valida al ramo (`"99"`) e asserisce il sollevamento di `UnknownValueException`. - **`testSerializationRoundTrip`**: serializza l'albero su file `.dmp` temporaneo mediante `salva()` e lo ricarica con `carica()`, verificando l'identità predittiva del modello deserializzato e la gestione di file inesistenti.

4.5 `server.SocketClientServerIntegrationTest`

Questa classe esegue test end-to-end su socket TCP: - **`setUpServer` / `tearDownServer`**: alloca un `ServerSocket` su porta effimera `0` ed esegue il dispatcher in un `ExecutorService`, garantendo chiusura immediata e

cancellazione dei file `.dmp`. - **`testFullFlowAcquireLearnPredict`**: client TCP che invia sequenzialmente i comandi `0` (acquisizione da DB `provac`), `1` (induzione e salvataggio `provac.dmp`), e `3` (predizione interattiva con risposta al bivio e ricezione dell'esito `OK` con valore predetto `10.0`). - **`testLoadArchiveAndPredictFlow`**: verifica la persistenza cross-client: il primo client acquisisce e crea il dump; il secondo client invia il comando `2` caricando il dump e procede alla predizione con esito corretto. - **`testErrorHandlingOverSocket`**: verifica che richieste di tabelle inesistenti restituiscano la stringa dell'eccezione, che caricamenti di file mancanti restituiscano `FileNotFoundException` e che comandi numerici sconosciuti (`999`) rispondano con `"Invalid command: 999"`.

4.6 `utility.KeyboardTest`

Questa classe valida la classe di utilità del client: - **`captureOriginalIn`** / **`restoreOriginalIn`**: preserva e ripristina lo standard input originale `System.in`. - **`provideInput`**: usa la Java Reflection per reimpostare i campi statici interni `in`, `reader` e `current_token` di `Keyboard`, azzerando i contatori di errore e garantendo l'assoluto determinismo di ogni test. - **`testReadStringAndWord`**: valida l'acquisizione di righe complete e token delimitati da spazi. - **`testReadBoolean`**: verifica il parsing insensibile al maiuscolo/minuscolo e il fallback `false` con incremento contatore errori per token errati. - **`testReadChar`**: legge singoli caratteri in sequenza. - **`testReadInt`**: effettua il parsing di interi positivi, negativi, zero e il fallback a `Integer.MIN_VALUE`. - **`testReadLong`**: effettua il parsing a 64 bit e il fallback a `Long.MIN_VALUE`. - **`testReadFloatAndDouble`**: verifica numeri decimali e il fallback a `Float.NaN` e `Double.NaN`. - **`testEndOfLineAndErrorControl`**: verifica la segnalazione di fine riga, il conteggio degli errori e il toggle `setPrintErrors`.

4.7 `server.EndToEndRestAndClientIntegrationTest`

Questa classe esegue test End-to-End (E2E) completi sul backend Spring Boot REST avviato su porta casuale (`RANDOM_PORT`): - **`testLearnTreeFromDatabaseE2E`**: invia una richiesta HTTP POST a `/api/trees/learn` con payload `{"tableName": "provaC"}`. Verifica la risposta HTTP 200 OK, la presenza dell'intestazione RULES, la correttezza del nodo radice `rootStep` (split sull'attributo `X` con 2 opzioni) e la creazione del file `provaC.dmp` su disco. - **`testPredictStepStatelessNavigationE2E`**: effettua una sessione di navigazione stateless completa: - Invoca `/predict` con percorso `[1]` (ramo `X=B`) e verifica il raggiungimento immediato di una foglia con valore numerico continuo predetto 10.0 e lista opzioni vuota. - Invoca `/predict` con percorso `[0]` (ramo `X=A`) e verifica il raggiungimento del nodo di split continuo `Y`. - Percorre i rami `[0, 0]` e `[0, 1]` verificando le rispettive foglie con valori continui stimati 1.0 e 1.5, coerenti con il comportamento del protocollo socket. - **`testLoadFromFileAndPredictE2E`**: dopo aver salvato il modello, invia una richiesta POST a `/api/trees/load` specificando `provaC.dmp`, verifica il corretto caricamento in memoria e la successiva capacità predittiva stateless. - **`testErrorHandlingE2E`**: verifica la conformità dei codici di stato e payload JSON d'errore: - `400 Bad Request` per tabelle inesistenti o nomi vuoti/nulli. - `404 Not Found` con messaggio descrittivo per archivi `.dmp` mancanti su disco. - `400 Bad`

`Request` per tentativi di navigazione su percorsi non validi (indici fuori limite o oltre nodi foglia). - **`testOpenApiAndSwaggerUiAvailabilityE2E`**: interroga l'endpoint `/v3/api-docs` verificando la presenza della specifica OpenAPI 3 con tutti gli endpoint documentati (`/api/trees/learn` , `/load` , `/predict`), e verifica che `/swagger-ui.html` e `/swagger-ui/index.html` rispondano con HTTP 200 OK erogando l'interfaccia interattiva.

4.8 `Client.TreeRestClientServiceTest`

Questa classe collauda in isolamento il client HTTP `TreeRestClientService` tramite `MockRestServiceServer`: - **`testLearnTreeSuccess`**: verifica la composizione della richiesta POST `/api/trees/learn` , l'header `Content-Type: application/json` e la deserializzazione del payload JSON in un record `TreeResponseDto` . - **`testLoadTreeSuccess`**: verifica la corretta invocazione di `/api/trees/load` e il popolamento del DTO di risposta. - **`testPredictStepSuccess`**: verifica l'invio della richiesta con percorso vettoriale `path` a `/api/trees/predict` e la conversione nel record `PredictionStepDto` . - **`testErrorHandlingWithJsonMessage`**: simula una risposta HTTP 400 Bad Request con body `{"error": "Messaggio di errore"}` , accertando che il client estragga il messaggio descrittivo e sollevi un'eccezione `RuntimeException` con il testo esatto ricevuto.

4.9 `Client.TreeMermaidConverterTest`

Questa classe collauda in isolamento il convertitore e parser visuale delle regole dell'albero: - **`testParseRulesNominal`**: valida la scomposizione delle 3 regole di `provaC` estraendo condizioni atomiche (`X=A` , `Y<=2.0`), valori di classe stimati e formulazione IF-THEN. - **`testConvertToMermaidNominal`**: accerta che il generatore sintetizzi un grafo Mermaid `graph TD` conforme con nodi di split rettangolari `:::split` , nodi foglia ellittici `:::leaf` ed etichette di bivio su ciascun arco orientato. - **`testSingleLeafTree`**: verifica il caso limite di un dataset degenere senza partizioni, producendo un diagramma con la sola foglia e valore stimato. - **`testEmptyOrNullInput`**: convalida la robustezza nei confronti di stringhe nulle o vuote restituendo una struttura controllata senza sollevare eccezioni. - **`testExtractCleanRules`**: verifica la produzione della lista formattata di regole leggibili utilizzata dal tab delle card visuali.

---

5. Modalità di Esecuzione dei Test

Per eseguire l'intera suite di test automatizzata (60 test) con Maven Surefire:

```
mvn clean test
```

Per eseguire selettivamente i test del server o del client:

```
# Modulo Server (44 test: dati, database, albero, socket ed E2E REST)
mvn -pl Server test

# Modulo Client (11 test: console keyboard e client REST HTTP)
mvn -pl Client test
```

Per eseguire una singola classe di test specifica (ad esempio `EndToEndRestAndClientIntegrationTest`):

```
mvn -pl Server test -Dtest=EndToEndRestAndClientIntegrationTest
```

---

6. Report di Esecuzione e Metriche di Copertura

Tutti i test inclusi nella suite vengono eseguiti con successo senza alcun fallimento né errore (**0 failures, 0 errors**):

Modulo	Classi di Test	Test Totali	Esito	Tempo Esecuz Tipic
Root	Aggregatore POM	0	SUCCESS	~0.2
Client	`KeyboardTest`, `TreeRestClientServiceTest`, `TreeMermaidConverterTest`	16	SUCCESS (0 Failures, 0 Errors)	~1.3
Server	`AttributeTest`, `DataTest`, `DatabasePackageTest`, `TreeNodesAndAlgorithmTest`, `SocketClientServerIntegrationTest`, `EndToEndRestAndClientIntegrationTest`	44	SUCCESS (0 Failures, 0 Errors)	~16.5
Totale Progetto	**9 Classi di Test**	**64**	**100% PASS**	**~18**